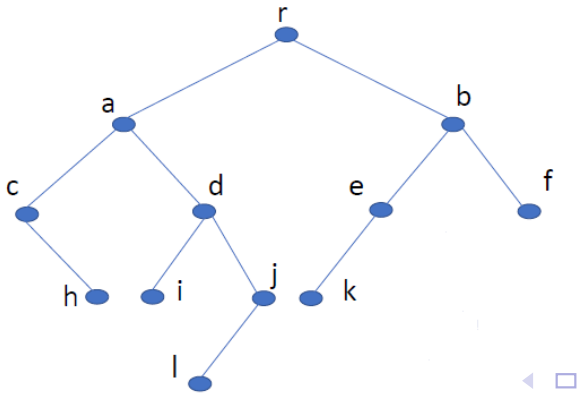
Recorrido de árboles binarios

Los árboles binarios ordenados se utilizan para almacenar información. Necesitamos procedimientos para *visitar* cada uno de sus vértices para acceder a los datos. Estudiaremos 4 tipos de recorridos: orden por nivel, pre-orden, post-orden e in-orden.

Definición

El **orden por nivel** de un árbol ordenado es un listado de los vértices en orden creciente según la profundidad, tal que los vértices que tienen la misma profundidad son listados acordes a su orden prescripto. De arriba hacia abajo y de izquierda a derecha.

Figura 1: Orden por nivel: r a b c d e f h i j k l



Definición

El **pre-orden izquierdo** de un árbol binario, se puede describir geométricamente así: hacemos una curva alrededor del árbol empezando por la raíz y dejando el árbol a la izquierda. A medida que vemos los vértices los vamos anotando en una lista.

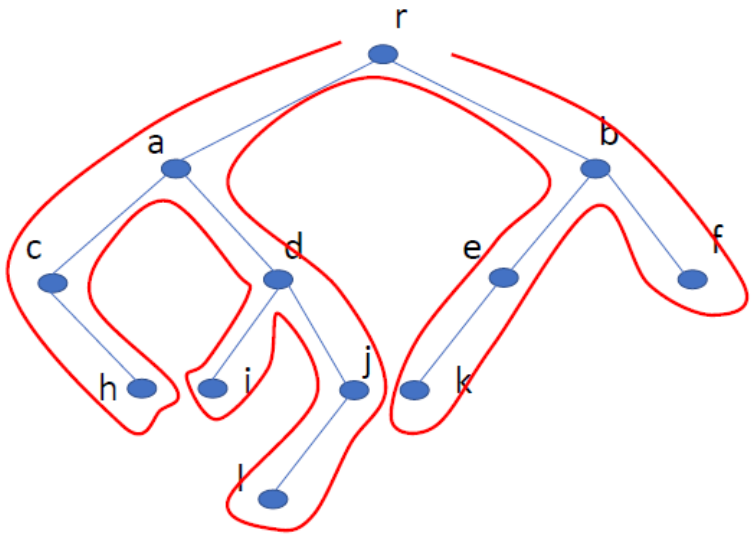


Figura 2: Pre-orden izquierdo: r a c h d i j l b e k f

Definición

El **post-orden izquierdo** de un árbol binario, se puede describir geométricamente así: hacemos una curva alrededor del árbol empezando por la raíz y dejando el árbol a la izquierda. Anotamos los vértices la última vez que los vemos.

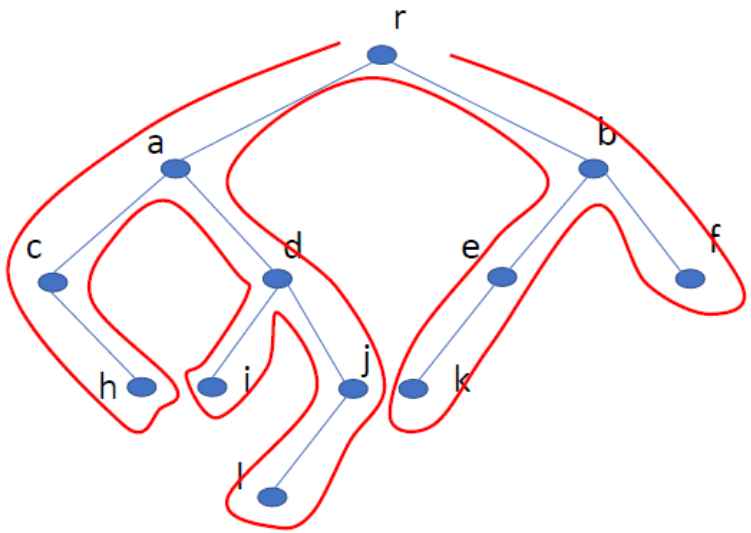


Figura 3: Post-orden izquierdo: h c i l j d a k e f b r

Definición

El **in-orden** de un árbol binario, se puede describir geométricamente así: hacemos una curva alrededor del árbol empezando por la raíz y dejando el árbol a la izquierda. Anotamos los vértices la segunda vez que los vemos.

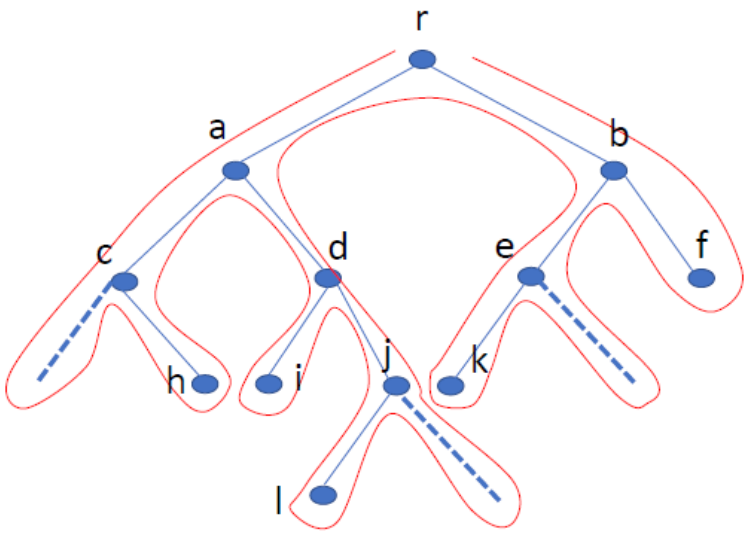


Figura 4: In-orden: c h a i d l j r k e b f

Árboles de expresión

Definición

Un **árbol de expresión** para una expresión aritmética es o un vértice etiquetado por un número o es un árbol que tiene un operador en la raíz y los subárboles izquierdo y derecho son árboles de expresión.

Ejemplo: Un árbol de expresión para la operación $((3+2)*(7-4))/8$

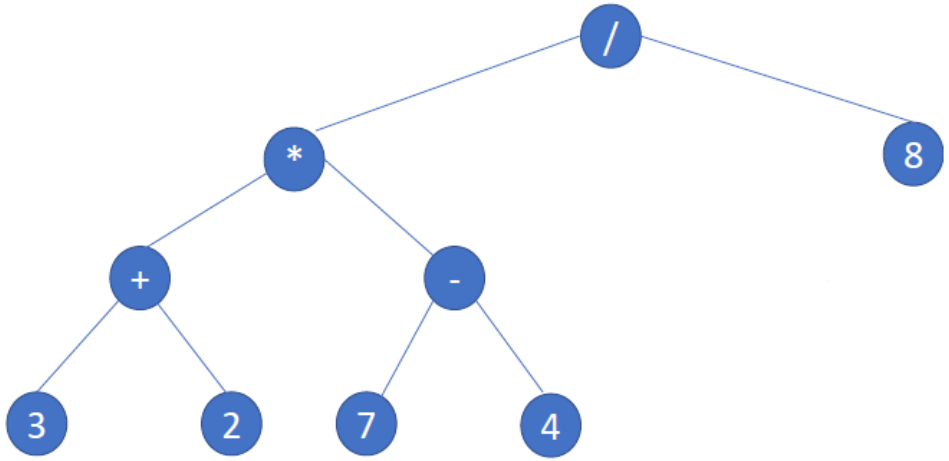


Figura 5: Las hojas son números, los vértices internos operaciones.

Notación Prefija, Postfija e Infija

Cuando un árbol de expresión se recorre, los vértices aparecerán en notación *Prefija, Postfija e Infija*, de acuerdo a qué recorrido se use.

Ejemplo: En el árbol anterior:

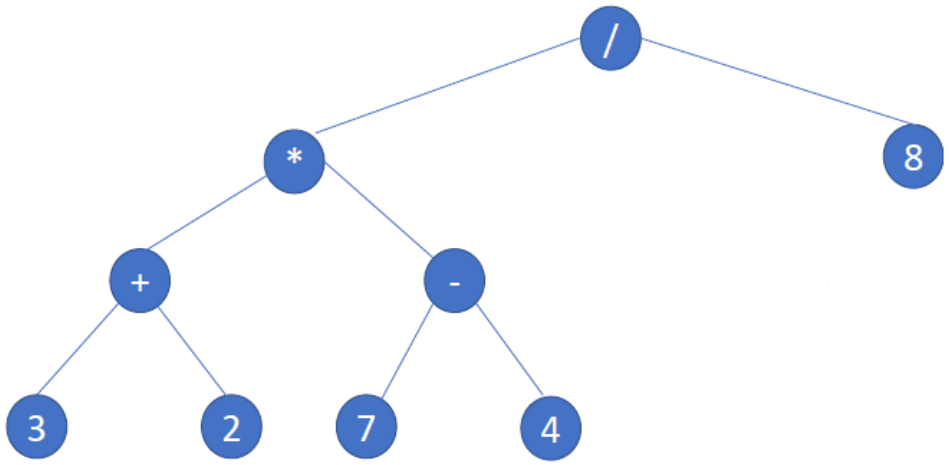


Figura 6

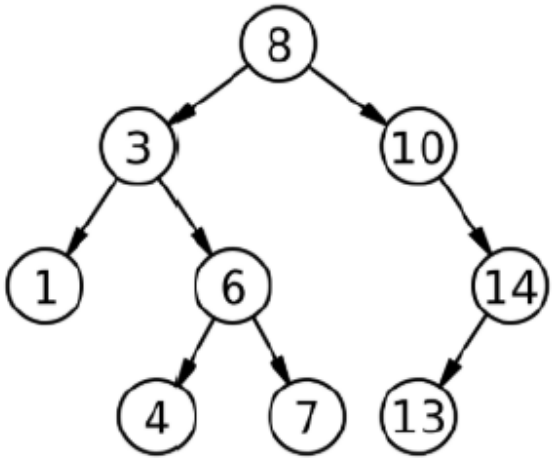
Notación prefija / * + 3 2 - 7 4 8
Notación postfija 3 2 + 7 4 -* 8 /
Notación infija ((3 + 2) * (7 - 4)) / 8
La notación infija requiere paréntesis.

Árboles binarios de búsqueda

Definición

Un **árbol binario de búsqueda (BST)** es un árbol binario donde cada vértice tiene asignado una clave tal que la clave de un vértice v es más grande que todas las de su subárbol izquierdo y más chica que todas las claves de su subárbol derecho.

Ejemplo: Buscamos $t = 7$:

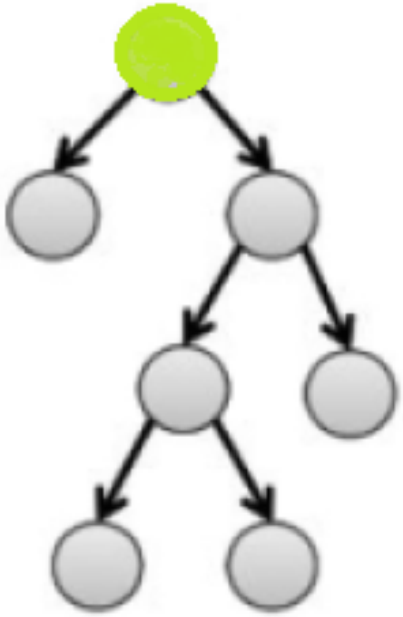


v
8
7<8
3
7>3
6
7>6
7
7=7

Definición

Un árbol binario está **balanceado** si para todo vértice el número de vértices de su subárbol izquierdo difiere a lo sumo en uno respecto al número de vértices de su subárbol derecho.

(a) Árbol no balanceado



(b) Árbol balanceado

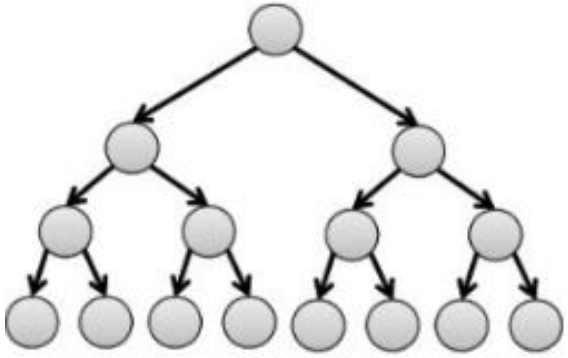


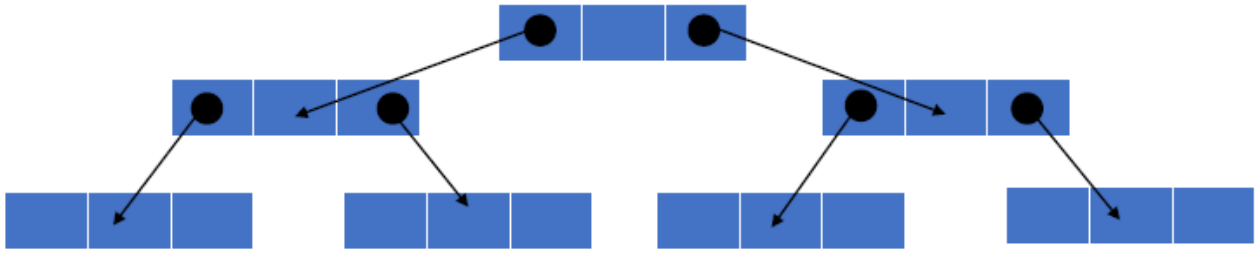
Figura 8: El primer árbol no está balanceado porque el subárbol izquierdo del vértice verde tiene un nodo y el derecho, tiene 5.

En base de datos las operaciones primarias son "**búsqueda**","**inserción**" y "**borrado**". Se presentan tres implementaciones de software diferentes:

1. Array de registros ordenado. (Estático) Tiene longitud fija.

2. Lista enlazada ordenada. (Dinámico) Tiene longitud variable.

3. Árbol binario de búsqueda. (Enlace dinámico)



Obs:

Realizar una búsqueda

1. requiere en el peor de los casos, $\mathcal{O}(\log_2 n)$ cálculos.
2. Tiene que ser secuencial a la fuerza entonces es de $\mathcal{O}(n)$.
3. Tiene orden $\mathcal{O}(\log_2 n)$, si el árbol está balanceado, si no lo está en el peor caso caemos en el punto 2), por ejemplo si ningún nodo tiene hijo derecho.

Obs:

Insertar o Borrar una clave:

1. Tiene orden $\mathcal{O}(n)$ porque hay que desplazar.
2. Tiene orden $\mathcal{O}(1)$.
3. Tiene orden $\mathcal{O}(1)$.

"Lo mejor" es la opción 3) pero siempre y cuando el árbol esté balanceado. Al insertar y/o borrar se va desbalanceando.

Ejemplo: Insertamos siempre como hoja. Es similar a la búsqueda.

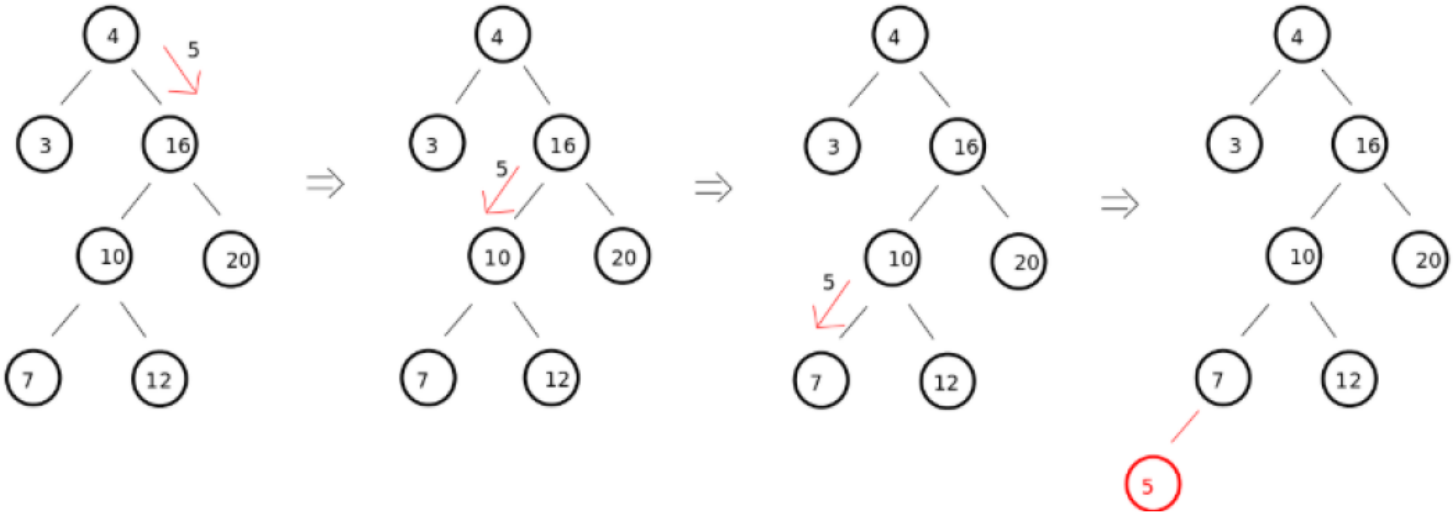


Figura 9: Insertar la clave 5.

Ejemplo: Caso 1: si el vértice es una hoja simplemente se borra.

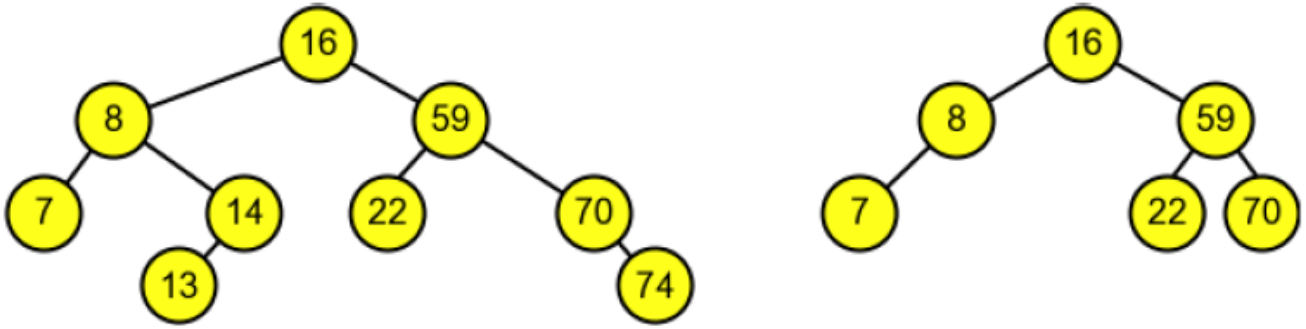


Figura 10: Borrar los vértices 13, 14 y 74.

Caso 2: Si el vértice tiene un único subárbol hijo, se borra y se reemplaza por éste.

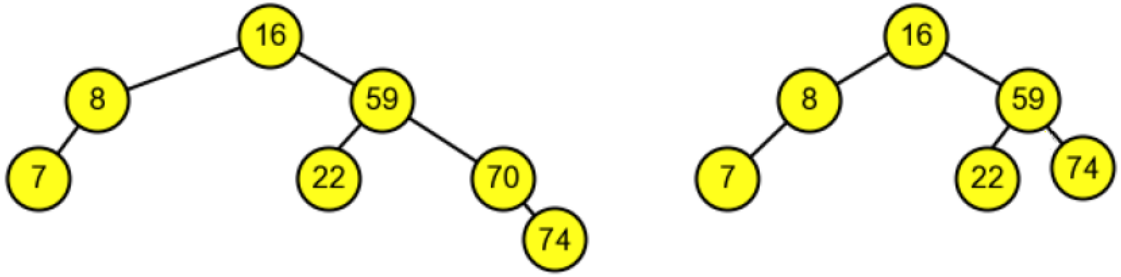
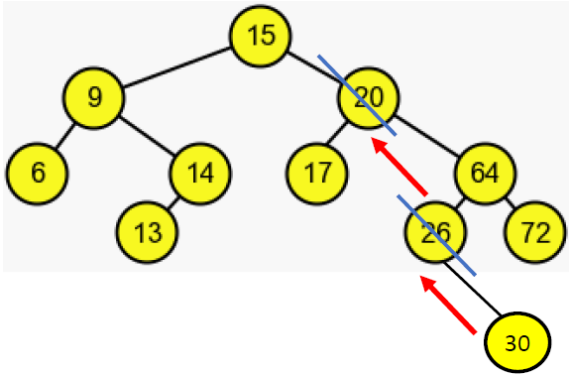


Figura 11: Borrar el vértice 70.

Caso 3: Si el vértice tiene dos hijos elegimos del subárbol derecho el vértice v con la menor clave. Reemplazamos el vértice a borrar por dicho vértice v y si ese vértice (v) tuviera un hijo derecho (izquierdo no puede tener) lo reemplazamos por su hijo derecho.



Del subárbol derecho del vértice 20, la menor clave es 26 reemplazamos el 20 por 26. Si el 26 tuviera un hijo derecho lo reemplazamos por su hijo.

Árboles recubridores

Supongamos que los caminos de una localidad están representados por el grafo simple de la Figura 12 a. En invierno, la única forma de mantener los caminos abiertos es barriéndolas con frecuencia. Se quiere limpiar la menor cantidad de caminos de forma tal que siempre haya caminos despejados conectando dos ciudades. Este problema se resuelve con un subgrafo conexo con el mínimo número de aristas y que contiene todos los vértices del grafo original (Figura 12 b).

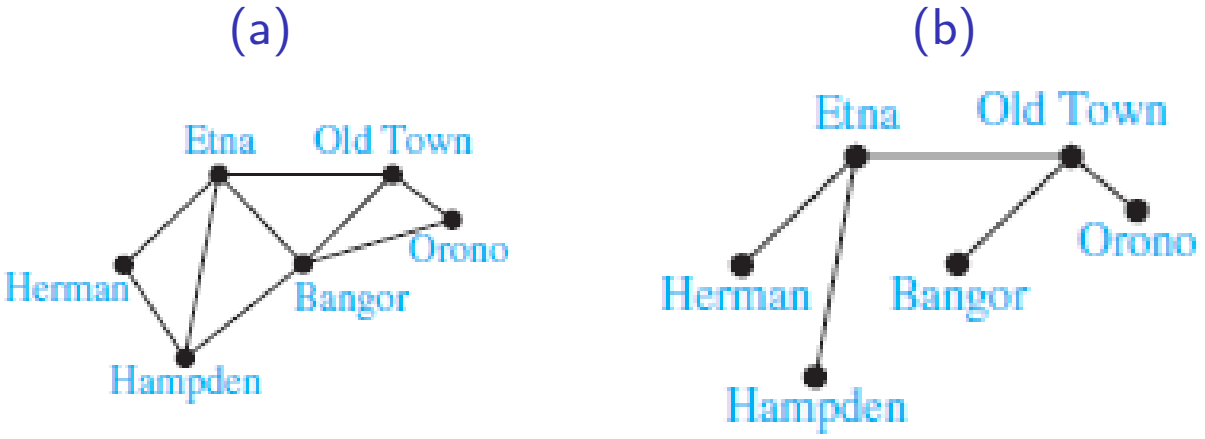


Figura 13

Definición

Se G un grafo simple. Un **árbol recubridor** de G es un subgrafo de G que es un árbol que contiene cada vértice de G.

Definición

Sea T un árbol subgrafo de G, una arista $e = \{a, b\}$ se llama **arista frontera de** T si a es un vértice de T y b es un vértice de G que no está en T.

Proposición

Sea T un árbol en un grafo G y sea e una arista frontera de T. Entonces $T + e$ es un árbol.

<u>Dem</u>: El agregado de dicha arista no genera ciclo y $T + e$ es conexo.

Algoritmo: Arbol recubridor

Entrada: grafo G conexo y v perteneciente a V_G.
Salida: Un árbol recubridor T.

1. Inicializar T como el vértice v;
2. etiqueta(v):=0; i:=1;
3. WHILE ($V_T \neq V_G$)
4. {
5. Elegir una arista frontera e de T, $e = \{u, w\}$ tal que $u \in V_T$ y w no pertenece a V_T.
6. Agregar w y e al árbol T
7. etiqueta(w):=i; i:=i+1;
8. }
9. RETURN T

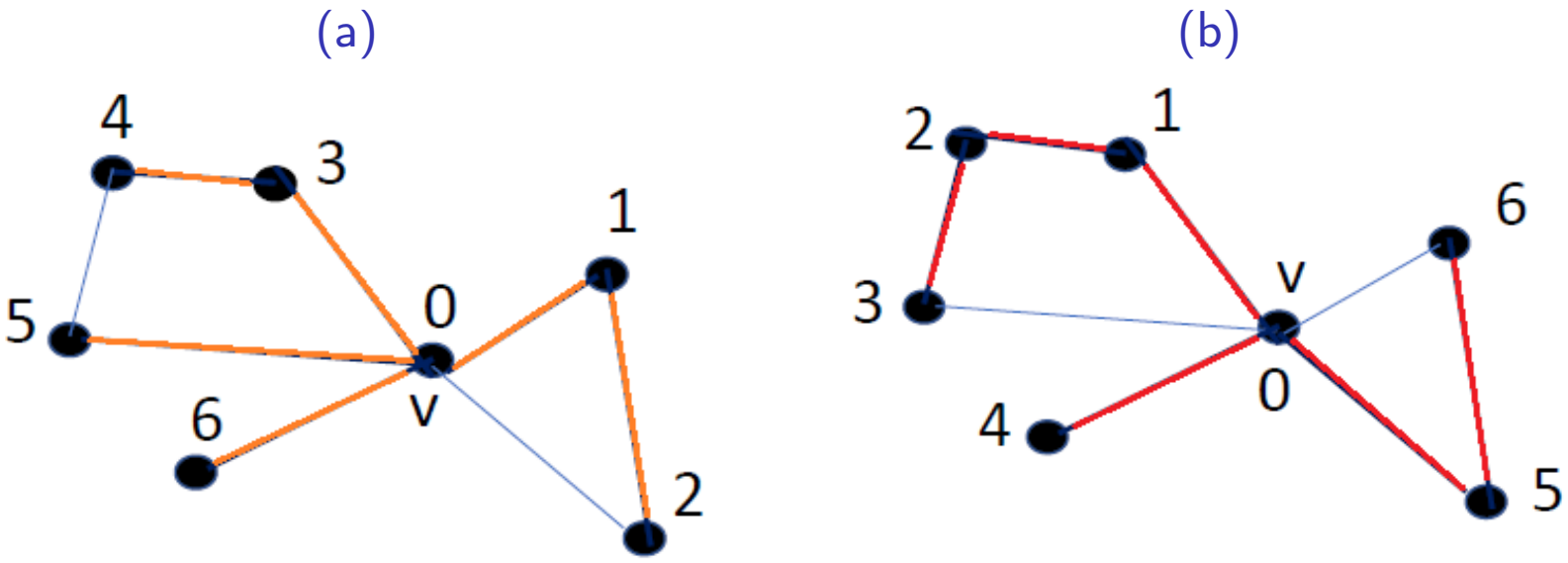


Figura 14: Dos árboles recubridores para el mismo grafo.

Depth-First Search (DFS)

O sea, recrrido en profundidad. Podemos usarlo para construir un árbol recubridor de un grafo simple G. Arbitrariamente se elige un vértice del grafo como raíz. Formamos un camino simple agregando sucesivamente vértices y aristas frontera. Se elige aquella cuyo vértice en el árbol fue *descubierto* último. Cuando no se puede, el algoritmo vuelve atrás e intenta de nuevo.

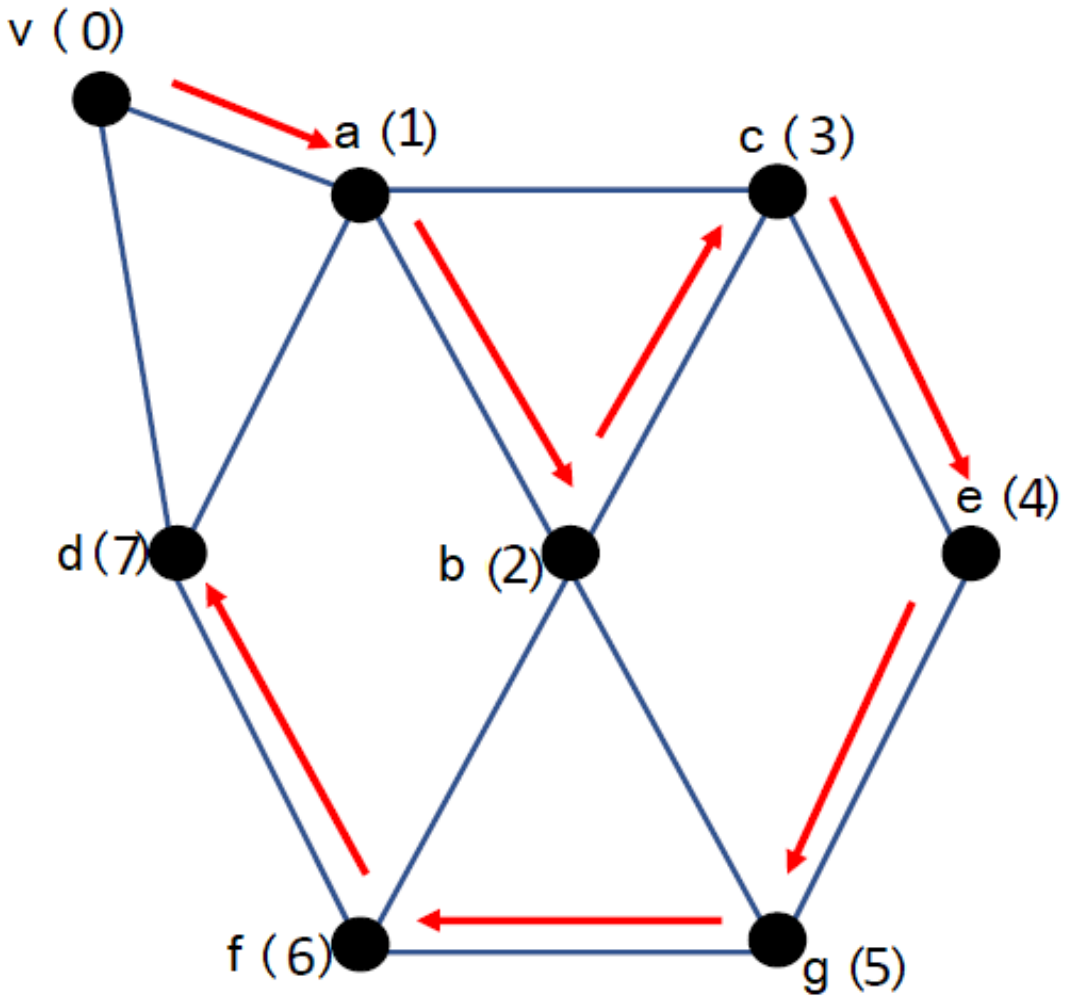
DFS Depth First Search

Algoritmo DFS

Entrada: G grafo conexo, v: vértice de G.
Salida: T árbol recubridor.

1. Inicializar T como el vértice v
2. $F := \emptyset$; dfnro(v)=0; i:=1;
3. WHILE ($V_T \neq V_G$)
4. {
5. $\quad F := \{e = \{v, w\} \ / \ v \in V_T \ \ y \ \ w \in V_G - V_T\}$
6. $\quad$ Sea e perteneciente a F y v tiene el mayor dfnro
7. $\quad$ Agregar w y e a T
8. $\quad$ dfnro(w):=i; i:=i+1;
9. }
10. RETURN T

Ejemplo:

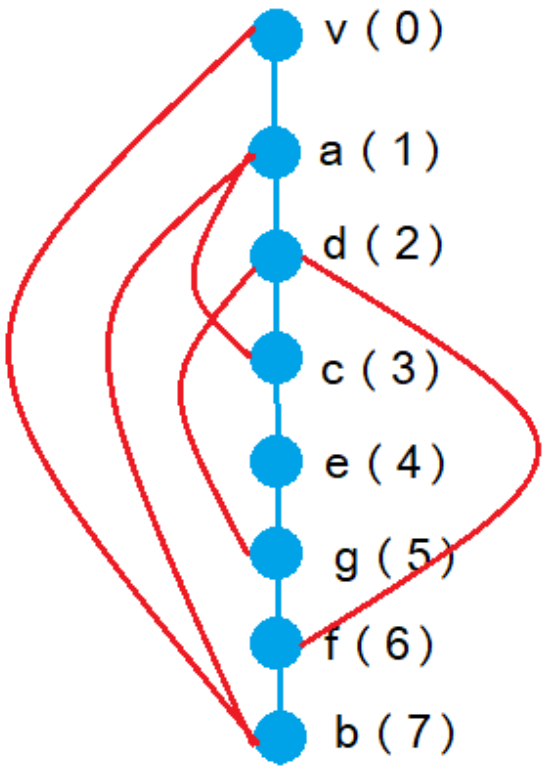


Nombre

La etiqueta de cada vértice v *descubierto* se llama $dfnro(v)$.

Proposición

Sea T un árbol recubridor producido por el algoritmo DFS en un grafo no dirigido y sea $e = \{x, y\}$ en $G - T$ tal que $dfnro(x) \leq dfnro(y)$ entonces x es un ancestro de y en T. (La arista e está en el grafo G pero no está en el árbol T).



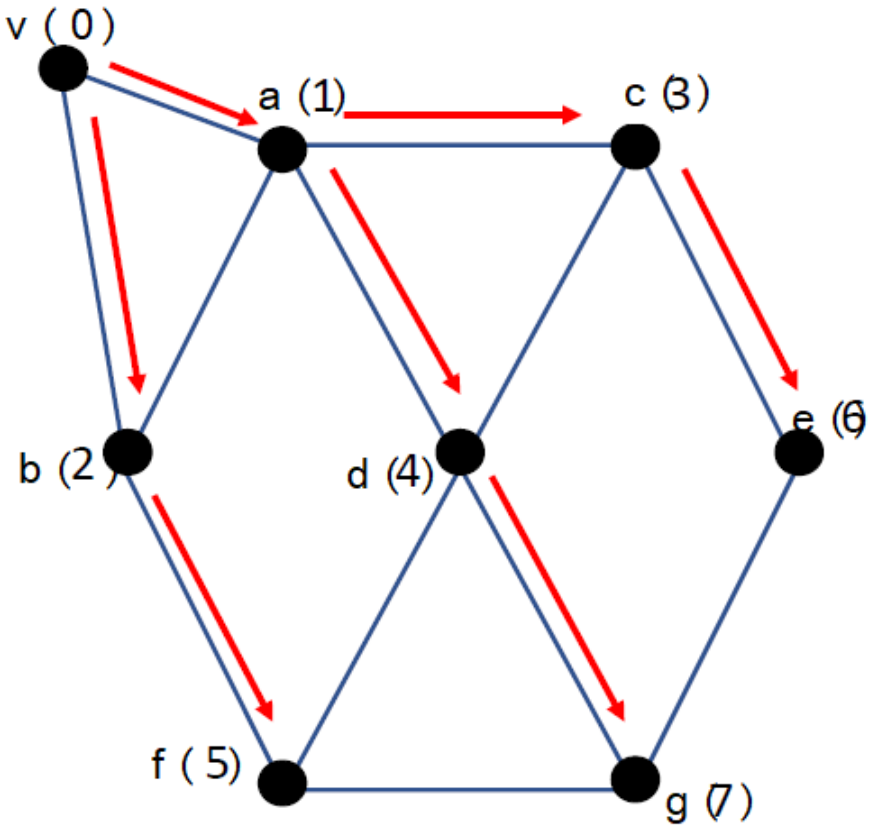
Breadth-First Search (BFS)

O sea, recorrido a lo ancho. A partir de un vértice arbitrario, se elige una arista frontera cuyo vértice en el árbol fue *descubierto* antes.

Algoritmo BFS

Entrada: G grafo conexo, v: vértice de G.
Salida: T árbol recubridor.

1. Inicializar T como el vértice v
2. $F := \emptyset$; etiqueta(v):=0; i:=1;
3. WHILE ($V_T \neq V_G$)
4. {
5. $\quad F := \{e = \{u, v\} \ / \ u \in V_T \ \ y \ \ v \in V_G - V_T\}$
6. $\quad$ Sea e perteneciente a F y u tiene menor etiqueta
7. $\quad$ Agregar v y e a T
8. $\quad$ etiqueta(v):=i; i:=i+1;
9. }
10. RETURN T

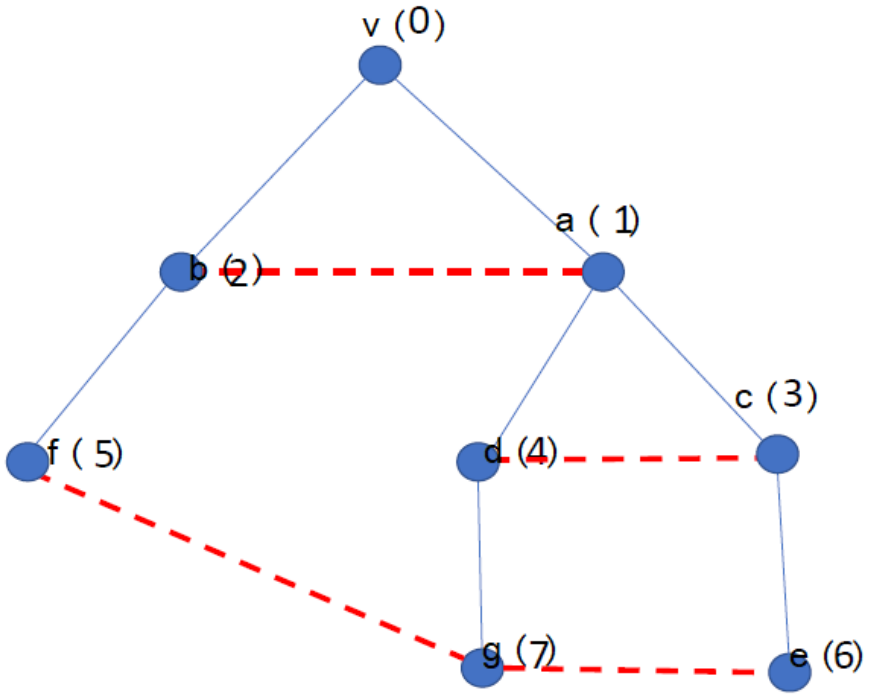


Obs.

Para tener unicidad de salida en los algoritmos **BFS** y **DFS** vamos a usar el orden lexicográfico cuando el algoritmo no decida que arista frontera elegir.

Proposición

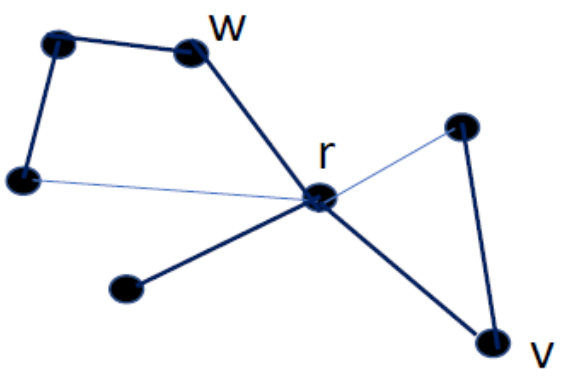
Cuando BFS es aplicado a un grafo no dirigido, los extremos de una arista en $G - T$ están al mismo nivel o en niveles consecutivos.



Proposición (*)

Sea T un árbol que resulta de aplicar DFS a un grafo G conexo, entonces la raíz r de T es un vértice de corte de G sii r tiene más de un hijo en T.

<u>Dem</u>: ($\Rightarrow$) Supongamos que r tiene sólo un hijo u en T todos los otros vértices son descendientes de $u \Rightarrow$ el subárbol que tiene como raíz a u es un árbol recubridor de $G - r \Rightarrow G - r$ es conexo $\Rightarrow r$ no es un vértice de corte.

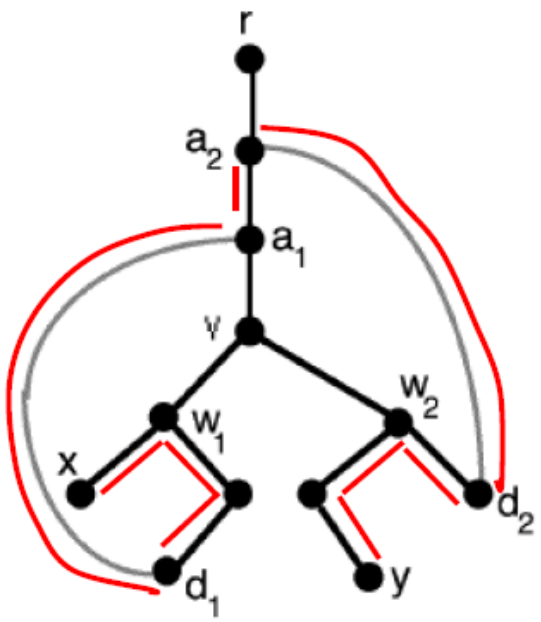


($\Leftarrow$) Supongamos que v y w son dos hijos de r, ninguno de los dos es ancestro del otro y ningún descendiente de v es ancestro o descendiente de algún descendiente de $w \Rightarrow$ por la proposición vista antes no hay en G arista que una el subárbol de v con el de $w \Rightarrow$ todos los caminos $v - w$ pasan por $r \Rightarrow r$ es vértice de corte.

Proposición (*)

Sea T un árbol que resulta de aplicar DFS a un grafo G conexo, entonces un vértice v de T que no es raíz es un vértice de corte de G sii v tiene un hijo w tal que ningún descendiente de w está unido a un ancestro propio de v por una arista en $G - T$.

Dem: ($\Rightarrow$) Supongamos que todo hijo w_i de v tiene un descendiente unido a un ancestro de v. Tomemos dos vértices x e y descendientes propios de v. Si x e y pertenecen al mismo subárbol w_i obviamente existe un camino que no contiene a v. Si x e y no son descendientes del mismo w_i, supongamos que x es descendiente de w_1 e y es descendiente de w_2, w_1 tiene un descendiente d_1 que está unido a a_1 que es un ancestro propio de v y w_2 tiene un descendiente d_2 que está unido a a_2 que es un ancestro propio de v, entonces existe un camino x y que no contiene a v.



($\Leftarrow$) Si v tiene un hijo w tal que ningún descendiente de w está unido a un ancestro propio de v por una arista en $G - T$, entonces todo camino $r - w$ debe pasar por v por lo que v es un vértice de corte.

Caracterización de los vértices de corte en términos de los dfnros

Definición

$menor(w) = \text{mín}\,\{dfnro(w), a\}$ siendo $a = \text{mín}\,\{dfnro(v_j)\}$, si existe en $G - T$ una arista $\{v_j, desc(w)\}$.

Proposición

Sea T el árbol resultante de aplicar DFS a un grafo conexo G. Un vértice v de T que no es la raíz es un vértice de corte $\Leftrightarrow$ v tiene un hijo w tal que $menor(w) \geq dfnro(v)$.

Ejemplo:

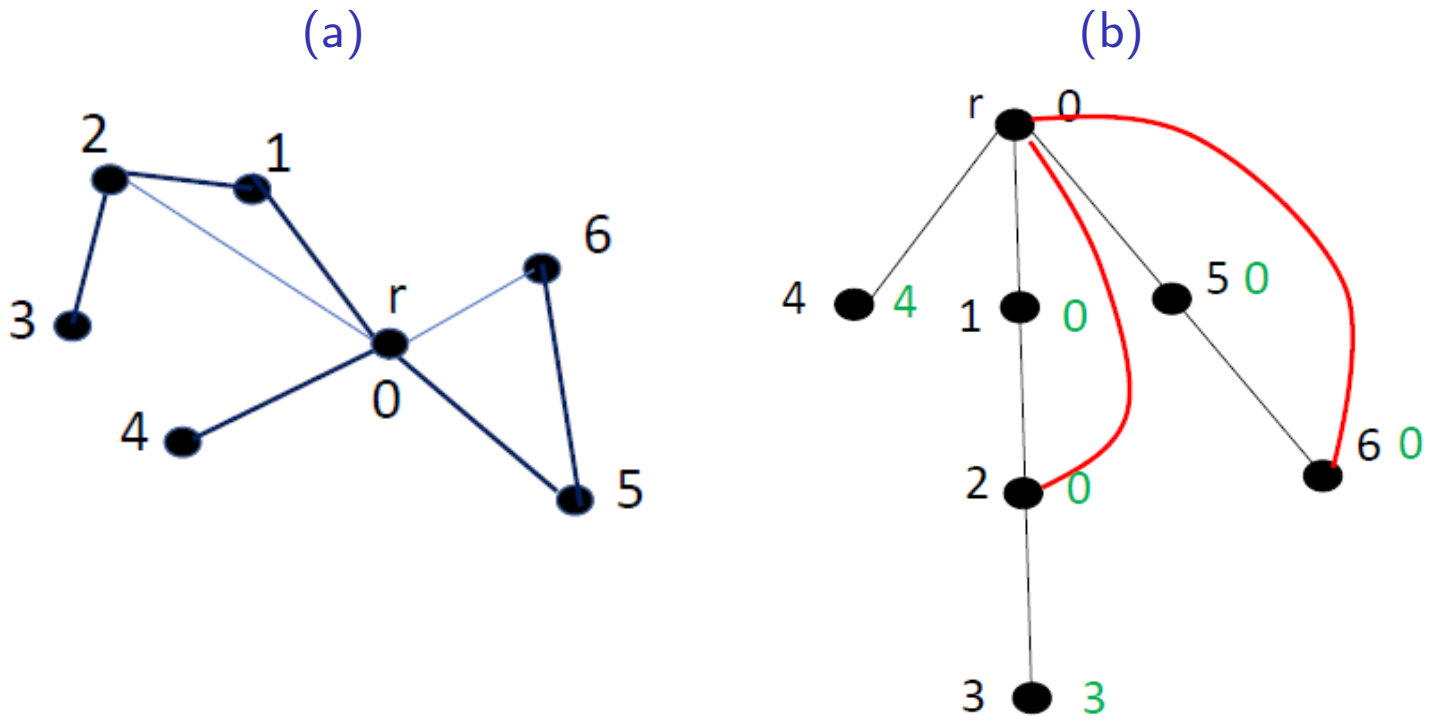


Figura 15: (a) Aplicamos DFS al grafo G (b) Árbol recubridor de G. Las aristas en $G - T$,están en rojo. Los menores, en verde.

- La raíz es vértice de corte porque tiene más de un hijo.
- El vértice cuyo dfnro es 2 es de corte ya que tiene un hijo cuyo menor es mayor o igual a su dfnro: $3 \geq 2$.

Algoritmo: Encontrando vértices de corte

Entrada: G grafo conexo con n vértices $\{v_1, \cdots, v_n\}$

Salida: Un conjunto K de vértices de corte de G

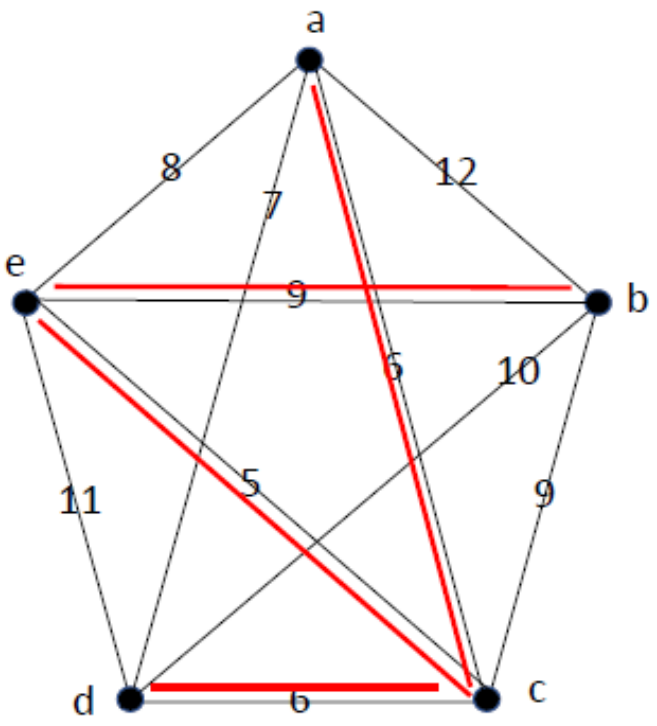
1. $K := \emptyset$.
2. Elegir $r \in V_G$ cualquiera.
3. Ejecutar DFS en G empezando por r.
4. Sea T el árbol recubridor.
5. IF (raíz(T) tiene más que un hijo)
6. THEN $K := K \cup \{$raíz(T)$\}$
7. FOR $I := 1$ TO n
8. { Calcular menor(v) }
9. FOR $I := 1$ TO n
10. {
11. IF ($v_i \neq r$ y existe un hijo w de v_i /menor(w)$\geq$dfnro(v_i))
12. THEN $K := K \cup \{v_i\}$
13. }
14. RETURN K

Árboles recubridores mínimos

Sea G un grafo conexo ponderado encontrar un **árbol recubridor mínimo** de G tal que el total de la suma de los pesos de las aristas sea mínimo.

Ejemplo: Algoritmo de Prim

(a)



(b)

Arista	Peso
{e,c}	5
{d,c}	6
{a,c}	6
{e,b}	9
Peso total	26

Figura 16: En cada paso obtenemos un subgrafo de G que es un árbol.

Algoritmo: Arbol recubridor de PRIM

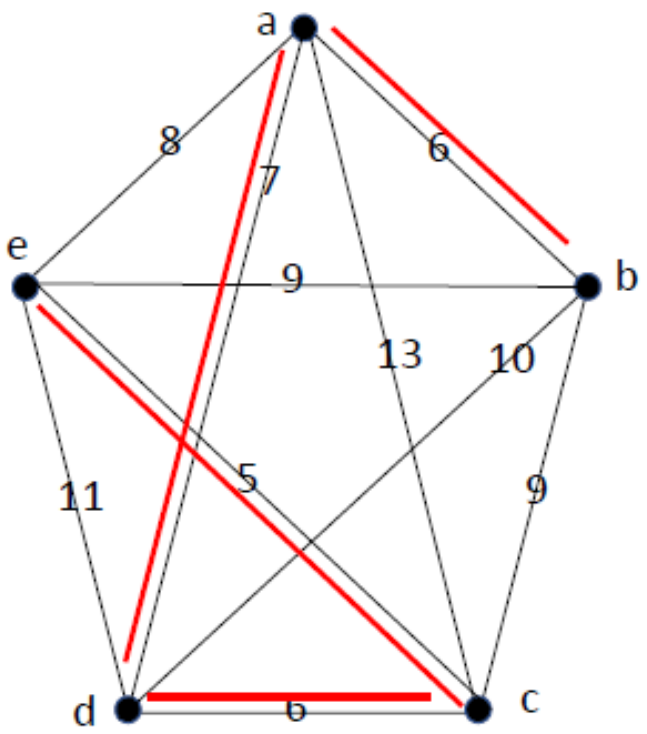
Entrada: G grafo conexo ponderado.
Salida: T árbol recubridor mínimo.

1. Elegir $s \in V_G$ cualquiera.
2. Inicializar T como el vértice s .
3. $F := \emptyset$.
4. WHILE ($V_G \neq V_T$)
5. {
6. $\quad F := \{e = \{v, w\} \ / \ v \in V_T \ \ y \ \ w \in V_G - V_T\}$
7. $\quad$ Sea $e \in F$ tal que e tiene el peso mínimo
8. $\quad$ Agregar w y e a T
9. }
10. RETURN T

Figura 17: La idea es comenzar en cualquier vértice y "hacer crecer"un árbol, agregando una arista frontera de peso más pequeño en cada iteración.

Ejemplo: Algoritmo de Kruskal

(a)



(b)

Arista	Peso
{e,c}	5
{a,b}	6
{d,c}	6
{d,a}	7
Peso total	24

Figura 18: En cada paso no se obtiene necesariamente un árbol.

Algoritmo: Arbol recubridor de Kruskal

Entrada: G grafo conexo.
Salida: T árbol recubridor mñimo.

1. I:=1
2. Sea e la arista en G con el menor peso.
3. $E_T := \{e\}$
4. WHILE (T no sea un árbol recubridor de G)
5. {
6. Sea $e \in E_G - E_T$ tal que tiene el mínimo peso y no forma ciclos con las aristas de E_T.
7. $E_T := E_T \cup \{e\}$
8. }
9. RETURN T

Figura 19: La idea es elegir una arista con el peso más chico y "hacer crecer" un árbol, agregando una arista frontera de peso mínimo en cada iteración.